Safety And Threat Analysis Tutorial

Writing Smart Contract Specification

To ensure reliability, security, and clarity in development and auditing, it's essential to begin with a well-structured and uambiguous DApp specification. Indeed, the specification is the mandatory entry point for safety and threat analysis.

DApp's Life Cycle

The specification must clearly describe the DApp's life cycle, including the deployment, interaction, upgrade (if applicable), and termination stages. This involves identifying:

- Deployment phase: Who deploys each contract? What are the initialization parameters?
- Operational phase: What interactions occur between users and the contract? What triggers state changes?
- Maintenance/upgrade (if allowed): Can contracts be upgraded? If so, how is trust maintained?
- Termination (if applicable): Under what conditions can the contract be terminated?

Each **script** may have its own individual life cycle and it's equally important to identify the actors who can interact with the contract.

Business Logic and Expected Behaviour for each Script

Each contract must have its business logic clearly defined. The use of controlled english is generally preferred not only to avoid any ambiguities or misinterpretations but also to be easily understandable by all the different actors involved in the development life cycle (e.g., developers, QA testors, formal methods engineer, auditors, etc). The business logic must outline:

- The contract's purpose and function
- The expected inputs and outputs to respectively be consumed and produced for each functional case (i.e., redeemer)
- All the assets that must be secured
- The set of invariants expected to be satsified
- The failure modes and how they should be handled.

This is where clarity becomes critical: the specification must act as a bridge between business requirements and technical implementation. Where possible, use state diagrams, flowcharts, or pseudocode to enhance clarity. This not only aids developers but also empowers non-technical stakeholders (e.g., product owners, legal reviewers) to understand the design.

Example: An Auction Smart Contract

The Plinth code for **AuctionValidator.hs** (https://github.com/IntersectMBO/plinth-template/blob/main/src/AuctionValidator.hs) is used as an example to show how a well-structured specification must look like. The specification follows faithfully what has been implemented and therefore may exhibit some security vulnerabilites / design flaws. These will be addressed in Section **Safety And Threat Analysis**.

The purpose of the `Auction` contract is to allow anyone to auction some assets (e.g., NFT token) that they own. Upon

deployment, the `Seller` will have to specify:

- A minimum bid amount
- The asset being auctioned; and
- A deadline for placing bids.

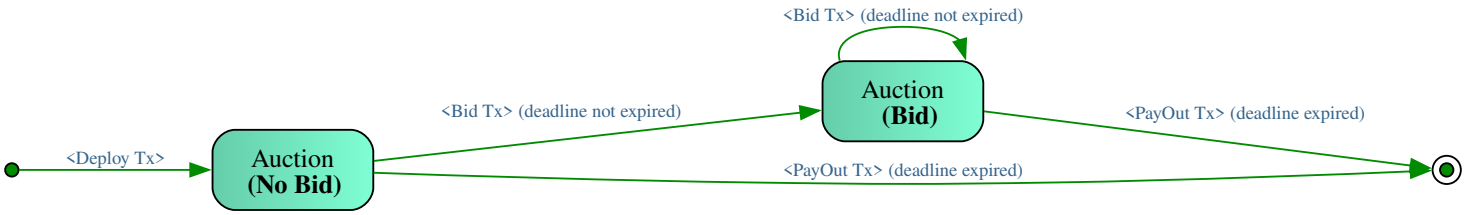

*Figure 1. **Auction Contract Life Cycle***

As shown in Figure ***Auction Contract Life Cycle***, `Bidders` may afterwards interact with the contract to make a formal offer for the auctioned asset. In particular, the initial bid must correspond at least to the minimum bid amount. Any new bid be must be higher than the previous one. Once a bid is made, the previous highest bid (if exists) is immediately refunded. When the bidding deadline has expired:

- New bids are no longer accepted;
- The asset can be transferred to the highest `Bidder` (if there is at least one bid); and
- The highest bid (if exists) can be transferred to the `Seller`.

Auction Deployment

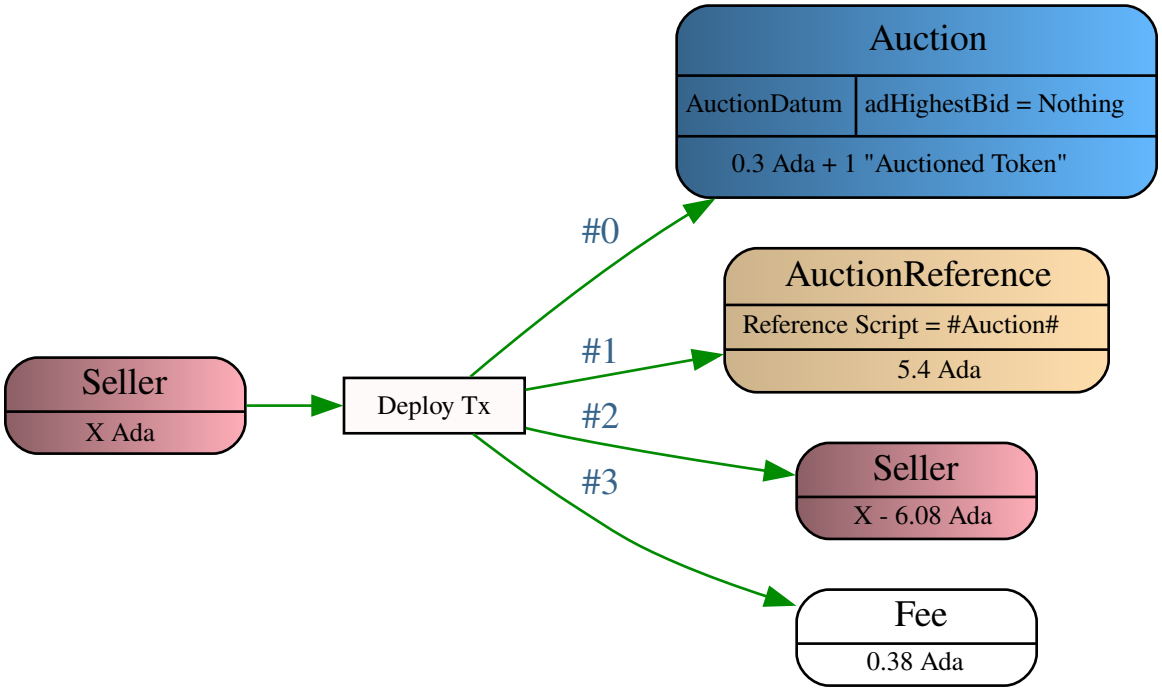

*Figure 2. **Auction Deployment Transaction***

When the `Auction` contract is first deployed, the `Seller` has to instantiate the contract with the following parameters:

- `apSeller` : public key hash that will be used to sent the highest bid / auctioned asset. This will generally correpond to the seller's public key hash but is not mandatory.

- `apCurrencySymbol` : currency symbol of the token being auctioned.
- `apTokenName` : name of the token being auctioned.
- `apMinBid` : The minimum bid in Lovelace.
- `apEndTime` : The bidding deadline.

The following *inline datum* needs to be present at the utxo produced for the `Auction` script:

```
Bid =
  { bAddr   :: BuiltinByteString
  , bPkh    :: PubKeyHash
  , bAmount :: Lovelace
  }

AuctionDatum {adHighestBid :: Maybe Bid}
```

There is not specific verification applied on deployment. However, a utxo is expected to be produced for the `Auction` script such that:

- `adHighestBid` is set to `Nothing` **AND**
- A `Auctioned Token` , identified by `apCurrencySymbol` and `apTokenName` , is present at the produced utxo.

Moreover, the use of *reference script* is optional when interacting with the `Auction` contract (see ***Auction Deployment Transaction***, **Bidding** and **PayOut**).

Bidding

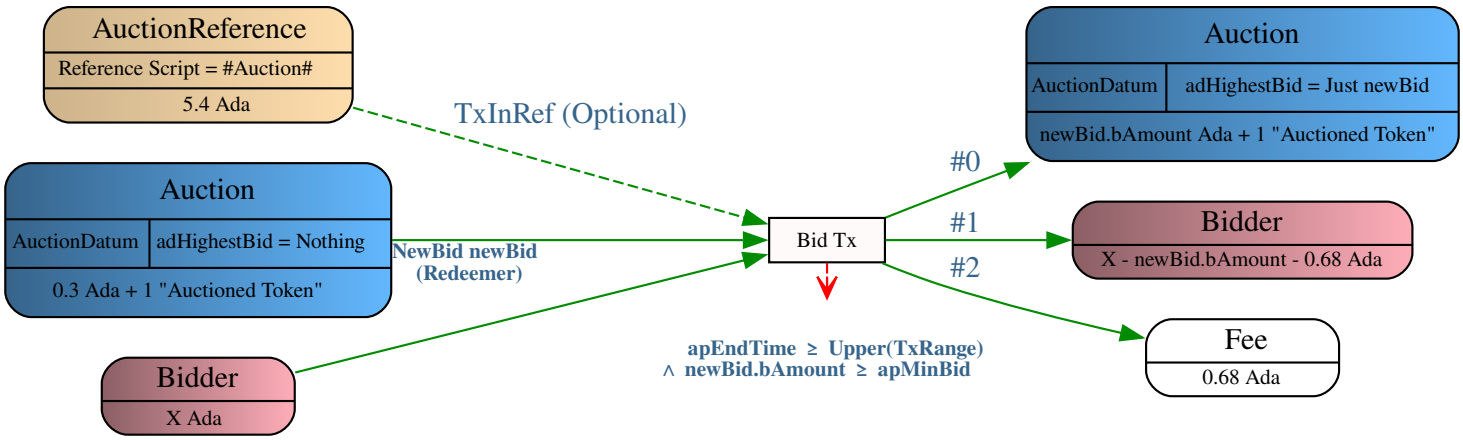

Figure 3. ***First Bidding Transaction***

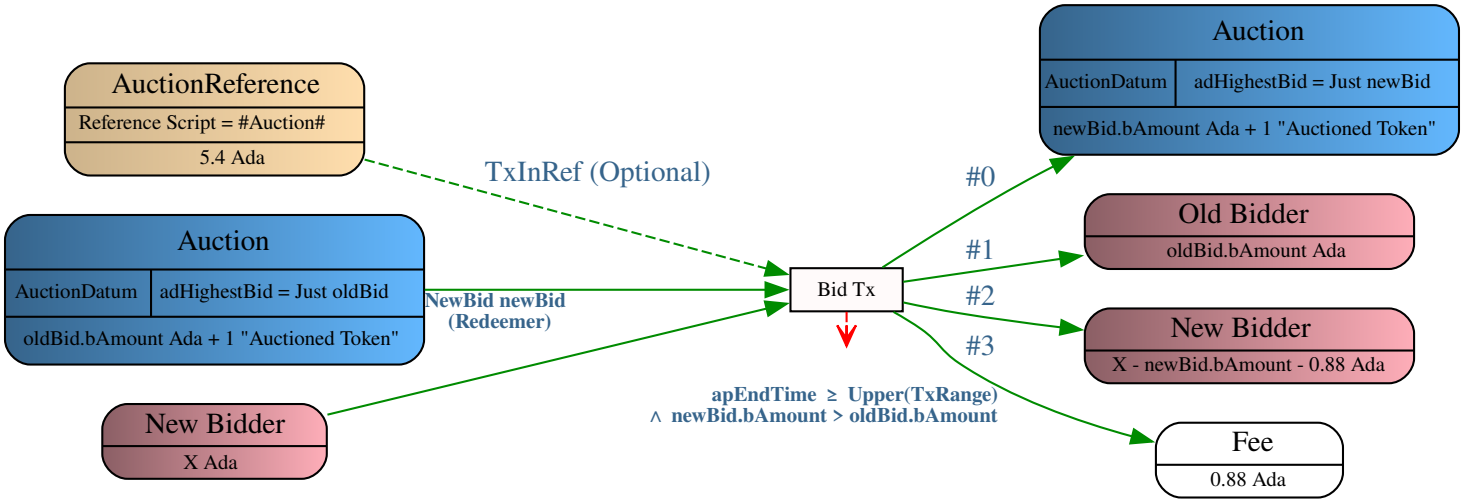

Figure 4. ***Next Bidding Transaction***

Once the `Auction` contract has been deployed, `Bidders` can make a formal offer for the auctioned asset as long as the bidding deadline has not expired. This can be performed using redeemer `NewBid`, which accepts an instance of `Bid` as argument. In particular, the `Auction` script's utxo can be consumed with redeemer `NewBid newBid` only when the following conditions are satisfied:

- **Let** `auction_datum` be the datum argument for the spending validator being evaluated **AND**
- `apEndTime` ≥ transaction's validity upper bound **AND**
- A utxo is produced for the current spending validator such that:
  - **Let** `auction_output` be the *inline datum* at the produced utxo **AND**
  - `auction_output.adHighestBid = Just newBid` **AND**
  - `Ada at produced utxo = newBid.bAmount` **AND**
  - Extacly one `Auctioned Token` is present at the produced utxo.
- **When** `auction_datum.adHighestBid = Nothing`:
  - `newBid.bAmount ≥ apMinBid` **AND**
- **When** `auction_datum.adHighestBid = Just oldBid`:
  - `newBid.bAmount > oldBid.bAmount` **AND**
  - A utxo is produced at an address with public key credential set to `oldBid.bPkh` such that:
    - `Ada at produced utxo = oldBid.bAmount`

PayOut

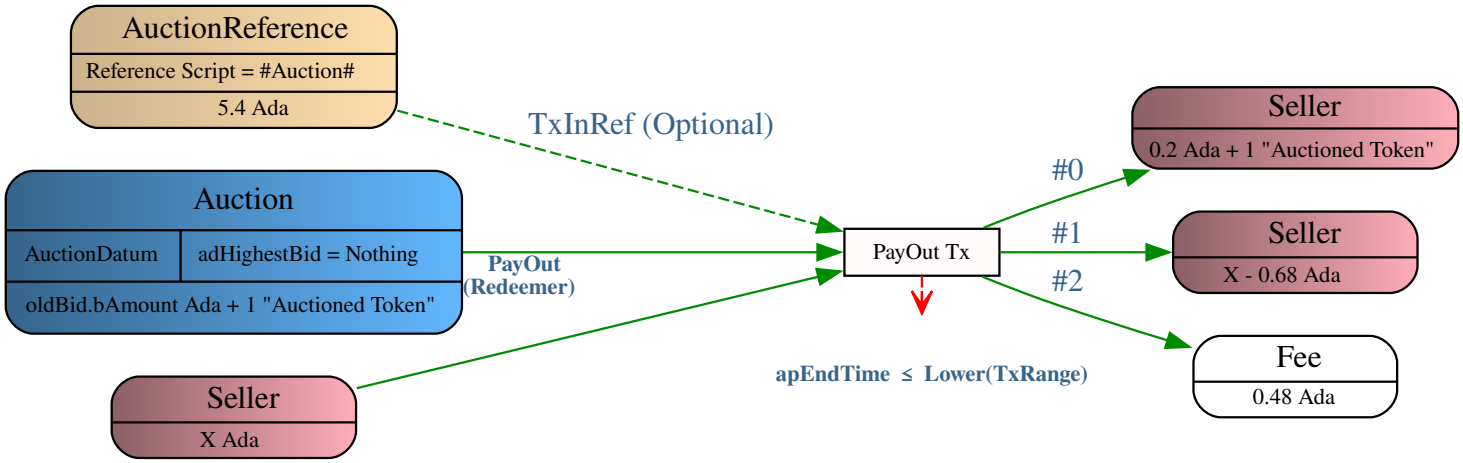

Figure 5. ***No Bid PayOut Transaction***

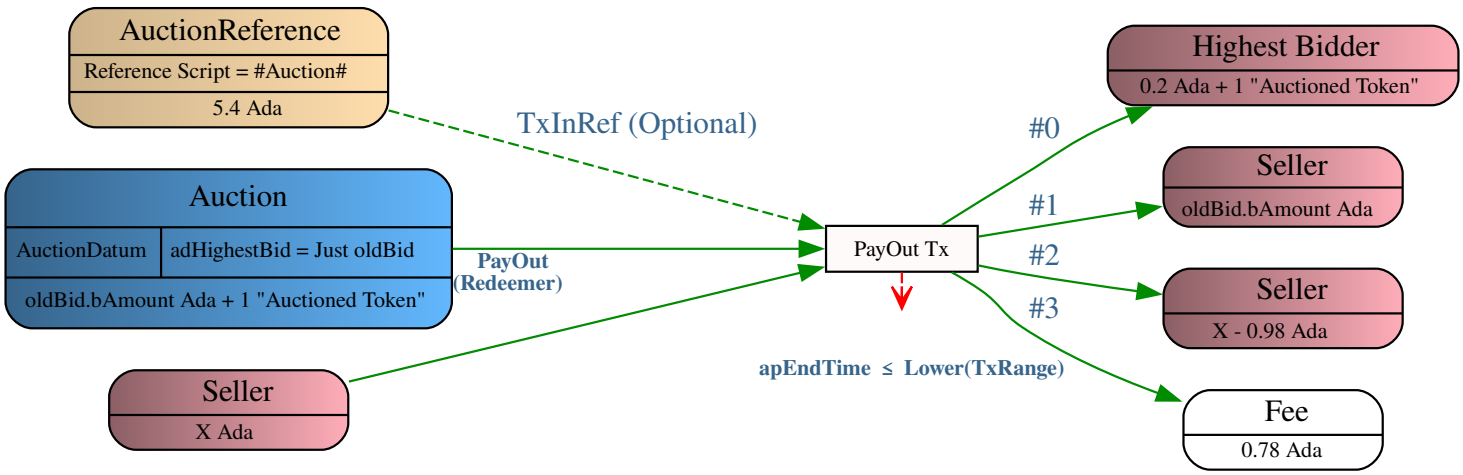

Figure 6. ***Bid PayOut Transaction***

Once the bidding deadline has expired, the only possible interaction with the `Auction` contract is to settle the bid via the `PayOut` redeemer. In this case, the `Auction` script's utxo can be consumed only when the following conditions are satisfied:

- **Let** `auction_datum` be the datum argument for the spending validator being evaluated **AND**
- `apEndTime` ≤ transaction's validity lower bound **AND**
- When `auction_datum.adHighestBid = Nothing`:
  - A utxo is produced at an address with public key credential set to `apSeller` such that:
    - Extacly one `Auctioned Token` is present at the produced utxo.
- When `auction_datum.adHighestBid = Just oldBid`:
  - A utxo is produced at an address with public key credential set to `oldBid.bPkh` such that:
    - Extacly one `Auctioned Token` is present at the produced utxo.
  - A utxo is produced at an address with public key credential set to `apSeller` such that:
    - `Ada at produced utxo = oldBid.bAmount`

Safety And Threat Analysis

The primary objective of safety and threat Analysis is to rigorously evaluate any specification to ensure that it:

- maintains correctness in all execution paths;
- has put in place the necessary mitigations to avoid any potential failure modes and attack vectors that can lead to unintended/undesired behaviours.

However, this activity requires a deep knowledge of the application domain and a critical-thinking mind. Hence, to properly conduct such an analysis on a DApp specification, one must not only have expertise in the smart contract language but must also be aware of the common security weaknesses pertinent to the language as well as the underlying blockchain.

The end result of this analysis is a clear set of properties that the specification must satisfy. These properties serve as validation criteria that the design includes appropriate mitigations to secure the application's logic and execution flow.

Failure Mode/Threat and Effect Analysis

Failure Mode and Effect Analysis (FMEA) is a systematic approach to identify, analyze and evaluate all the potential ways in which a system, protocol or component can fail. Here, the goal is to understand the causes and effects of each possible failure mode/attack vector so that appropriate mitigation strategies can be designed to reduce/eliminate the risk of undesired consequences. The key elemens for such an analysis are:

- **Failure Mode**: The specific way in which something might fail (e.g., incorrect output, unexpected error, security breach).
- **Cause**: The underlying reason for the failure (e.g., logic/arithmetic error, invalid input).
- **Effect**: The impact or consequence of the failure (e.g., protocol halting, token leakage, financial loss).
- **Severity**: Criticality assessment to prioritize risks (e.g., Low, Medium, High).

Common Plutus-Specific Weaknesses

The following is a list of common security weaknesses for Plutus, which can be used as a basis when conducting safety and treat analysis:

- **Large Value Attack**: Flooding the utxo of a spending script with a large number of tokens in order to trigger a denial-of-service condition (i.e., script budget exceeded).
- **Large Datum Attack**: Unprotected/Unbounded datum that can be exploited to trigger a denial-of-service condition (e.g., transaction's size limit/script budget exceeded).
- **Large Reference Script Attack**: Triggering a denial-of-service condition (i.e., transaction's size limit exceeded) by adding a large reference script to the utxo of a spending script (applicable as from ***Plutus V2***).
- **Multiple Satisfaction**: Multiple spending scripts utxos being consumed without satisfying the condition under which the total value can be spent. This can lead to unexpected token leakage or unauthorized actions.
- **Staking Credential Attack**: Anyone can deliberately change the staking credential at a spending script utxo. This can lead to unxpected disruption of reward distribution.
- **UTxO Contention**: When two (or more) transactions try to consume the same utxo. This can lead to protocol stalling.

Example: An Auction Smart Contract

Reference	Failure Mode/ Attack	Effect	Severity	Mitigation	Action
SAA-0001	`adHighestBid` set to `Just newBid` on deployment such that: • `newBid.bAmount > Ada at produced utxo`	First bidder can still make a formal offer if they are willing to: • bid more than `apMinBid` • provide the surplus of Ada necessary to refund `oldBid.bAmount` to `oldBid.bPkh` Seller will maciliously acquire extra Ada.	High	Must ensure that `adHighestBid` is set to `Nothing` on deployment	This mitigation can only be enforced via an NFT minting policy associated to the `Auction` script.
SAA-0002	`adHighestBid` set to `Just newBid` on deployment such that: • `newBid.bAmount < Ada at produced utxo`	• No one will be able to make a formal offer as the `Auction` script will always fail. • The `Auction` script will also fail upon a `PayOut` action as `newBid.bAmount` is less than the minimum Ada value required to transfer the `Auction token` back to the seller. Funds and Auctioned Token locked forever. Only the Seller will be penalized	Medium	Must ensure that `adHighestBid` is set to `Nothing` as mitigation against offchain code bugs.	This mitigation can only be enforced via a NFT minting policy associated to the `Auction` script.

SAA-0003	`adHighestBid` set to `Just newBid` on deployment such that: • `newBid.bAmount = Ada at produced utxo`	• First bidder can still make a formal offer if they are willing to bid more than `apMinBid` (i.e., only when `oldBid.bAmount > apMinBid`). • Seller will be refunded with `oldBid.bAmount` (e.g., min Ada value required when deploying the `Auction` script).	Low	Must ensure that `adHighestBid` is set to `Nothing` as mitigation against offchain code bugs.	This mitigation can only be enforced via a NFT minting policy associated to the `Auction` script.

SAA-0004	The `AuctionDatum` *inline datum* provided on deployment is unbounded such that: • extra fields, other than `adHighestBid`, are introduced at the `BuiltinData` representation level. Case when adHighestBid ≠ Nothing is already covered by the previous failure modes.	Case 1: The utxo can't be spent by any transaction due to size limit exceeded: • Only Seller is penalized ⇒ Funds and Auctioned Token locked forever. Case 2: The utxo can be spent by a Bid/PayOut transaction: • Bidder will have to pay for extra Tx fees, which can be significant according to the datum's size. • Bidder can save on Ada to be provided for corresponding bidding amount (i.e., `newBid.bAmount` including the min Ada value provided on deployment) ⇒ Seller penalized.	High	Must ensure that `AuctionDatum` is strictly bounded on deployment.	This mitigation can only be enforced via a minting policy associated to the `Auction` script.

SAA-0005	A large *reference script* is present at produced utxo on deployment.	Case 1: The utxo can't be spent by any transaction due to size limit exceeded: • Only Seller is penalized ⇒ Funds and Auctioned Token locked forever. Case 2: The utxo can be spent by a Bid/PayOut transaction: • Bidder will have to pay high Tx fees (i.e., *reference script* embedded in the Tx's script map). • Bidder can save on Ada to be provided for corresponding bidding amount (i.e., `newBid.bAmount` including the significant Ada value required on deployment) ⇒ Seller penalized.	High	Must ensure that no *reference script* is present at the produced utxo on deployment.	This mitigation can only be enforced via a minting policy associated to the `Auction` script.

SAA-0006	The `Auctioned Token` is not present at the produced utxo upon deployment	• No one will be able to make a formal offer as the `Auction` script will always fail. • The `PayOut` action will succeed only when the Seller is able to provide `Auctioned Token`. Otherwise, Funds locked forever.	Medium	Must ensure that `Auctioned Token` is provided on deployment as mitigation against offchain code bugs.	This mitigation can only be enforced by: • using an NFT minting policy associated to the `Auction` script; **AND** • keeping parameters `apCurrencySymbol` and `apTokenName` in the `AuctionDatum`.
SAA-0007	Other tokens (apart from the Auctioned Token and the NFT Token) are present at the produced utxo upon deployment.	First Bidder can acquire the extra tokens when making a formal offer ⇒ Seller penalized.	Medium	Must ensure that no other tokens, other than the `Auctioned Token` and NFT Token, are provided upon deployment as mitigation against offchain code bugs.	This mitigation can only be enforced by: • using a NFT minting policy associated to the `Auction` script; **AND** • keeping parameters `apCurrencySymbol` and `apTokenName` in the `AuctionDatum`.

SAA-0008	Upon deployment, `Ada at produced utxo > apMinBid`. E.g., name of `Auctioned Token` correponds to the ByteString representation of a compressed artwork ⇒ High min Ada value.	• Bidder will not even have to provide any Ada when making a formal offer that is less than `Ada at consumed utxo`. • Upon `PayOut` (i.e., bidding deadline expired), Seller will receive *nothing* as they will have to provide the min Ada value required to transfer the `Auctioned Token` to the highest Bidder. ⇒ Seller penalized.	High	Must ensure that `apMinBid ≥ 2 × Ada at produced utxo` on deployment.	This mitigation can only be enforced by: • using a NFT minting policy associated to the `Auction` script; **AND** • keeping parameter `apMinBid` in the `AuctionDatum`.

SAA-0009	Upon deployment: • `Ada at produced utxo ≤ apMinBid` **AND** • Minimum Ada value *may be* quite high (e.g., artwork is quite significant) **AND** • First Bidder only provide newBid.bAmount - Ada at consumed utxo Ada when making a formal offer.	• Upon a next bid (if any): ◦ Next Bidder will provide `newBid.bAmount` Ada **AND** ◦ First Bidder will be refunded with more Ada than they initially offer. • Upon `PayOut`, Seller will be penalized twice as: ◦ They will receive less Ada than expected **AND** ◦ They will have to provide the minimum Ada value required to transfer the `Auctioned Token` to the highest Bidder. Seller will lose 2 × initial Minimum Ada Value First Bidder will acquire the initial Minimum Ada Value	High	Must force Bidder to provide `newBid.bAmount` in addition to the minimum Ada value provided on deployment.	This mitigation can only be enforced by: • using a NFT minting policy associated to the `Auction` script; **AND** • keeping parameter `apMinBid` in `AuctionDatum` **AND** • introducing field `minAdaValue` in `AuctionDatum` **AND** • ensuring that `Ada at produced utxo = minAdaValue + newBid.bAmount` for each new bid.

SAA-0010	Upon deployment: • `apEndTime` < transactions validity upper bound	• No one will be able to make a formal offer as the `Auction` script will always fail. • Seller will still be able to get back the `Auctioned Token` once bidding deadline has completely expired.	Low	Must ensure that `apEndTime` > transactions validity upper bound upon deployment as mitigation against offchain code bugs.	This mitigation can only be enforced by: • using a NFT minting policy associated to the `Auction` script; **AND** • keeping parameter `apEndTime` in `AuctionDatum`.

SAA-0011	Upon bidding (i.e., redeemer `NewBid` `newBid`), the provided `AuctionDatum` *inline datum* is unbounded such that: • extra fields, other than `adHighestBid` , are introduced at the `Builtin Data` representation level; **OR** • `newBid.bAddr` has a Builtin ByteString representation exceeding 32 bytes; However, `newBid.bPkh` is properly set for Bidder to receive `Auctioned Token` upon `PayOut` .	The large datum attack has been done such that: • The `Auction` utxo can't be spent by a `NewBid` transaction (i.e., size limit exceeded when producing the new utxo). **AND** • The `Auction` utxo can only be spent by a `PayOut` transaction. Protocol halted for any legitimate bid	High	Must ensure that for each new bid: • `AuctionDatum` is strictly bounded. **AND** • `newBid.bAddr` must be removed as not even used anywhere. **AND** • `apCurrencySymbol` , `apTokenName` , `apMinBid` , `minAdaValue` and `apEndTime` must remain unaltered.	This mitigation can only be enforced by: • removing `bAddr` from `Bid` data structure; **AND** • ensuring that: ◦ `auction_output` is strictly bounded such that it does not contain any extra fields **AND** ◦ `auction_datum.apCurrencySymbol = auction_output.apCurrencySymbol` **AND** ◦ `auction_datum.apTokenName = auction_output.apTokenName` **AND** ◦ `auction_datum.apMinBid = auction_output.apMinBid` **AND** ◦ `auction_datum.minAdaValue = auction_output.minAdaValue` **AND** ◦ `auction_datum.apEndTime = auction_output.apEndTim`

					`e` **AND** ◦ `NewBid.newBid.bPkh` is the only signatory (i.e., implicitly guarantees that bPkh is properly bounded). NOTE: NewBid.newBid.bAmount is implicitly bounded due to explicit verification on expected Ada at produced utxo
SAA-0012	Upon bidding (i.e., redeemer `NewBid newBid`), the new `Auction` utxo is flooded with additional tokens, each of which precedes both `Auction` NFT token and `Auctioned Token` in the `Value` map.	The `Auction` utxo can't no more be spent by any transaction due to script budget exceeded when trying to search for the `Auction` NFT token in the list of consuming utxos. Funds and Auctioned Token locked forever.	High	Must ensure that only the `Auction` NFT token and the `Auctioned Token` are present at the produced utxo upon deployment and upon any new bid.	Update specification to enforce mitigation.

SAA-0013	• Seller deploys two (or more) `Auction` instances for the same `Auctioned Token` (i.e., have more than one token representing the same artwork) **AND** • The same Bidder make a formal offer for all the `Auctioned Tokens`	Another Bidder makes a formal offer by consuming all the `Auction` utxo within the same transaction such that: • Only one `Auction` utxo, satisfying the bidding constraints, is produced **AND** • The previous Bidder is refunded only once (i.e., only one utxo is produced) **AND** • Current Bidder maliciously collects the remaining Auctioned Tokens and funds.	High	Must ensure that only one `Auction` NFT token is minted upon deployment for each `Auction` instances.	This mitigation can only be enforced by: • using a NFT minting policy associated to the `Auction` script; **AND** • parameterizing the `Auction` contract with the utxo used to mint the NFT token **AND** • keeping the currency symbol of the minting policy in `AuctionDatum` to validate: ◦ The presence of the NFT token upon each new bid **AND** ◦ The burning of the NFT token upon `PayOut`.

Last updated 2025-07-24 19:45:32 +0200